

Appendix E: Errata

Alphabetical by section.

ACCESSCTRL

RP2350-E3

Reference	RP2350-E3
Summary	In QFN-60 package, <code>GPIO_NS_MASK</code> controls wrong <code>PADS</code> registers
Affects	RP2350 A2, QFN-60 package only
Description	RP2350 remaps IOs, their control registers and their ADC channels so that both package sizes appear to have consecutively numbered GPIOs, even though for physical design reasons the QFN-60 package bonds out a sparse selection of IO pads. The connection between the <code>GPIO_NS_MASK0</code>/<code>GPIO_NS_MASK1</code> registers and the <code>PADS</code> registers does not take this remapping into account. Consequently, in the QFN-60 package only, the <code>GPIO_NS_MASK0</code> register bits are applied to registers for the wrong pads. Specifically, <code>PADS_BANK0</code> registers 29 through 0 are controlled by the concatenation of <code>GPIO_NS_MASK</code> bits 47 through 44, 39 through 33, 30 through 28, 24 through 17 and 15 through 8 (all inclusive ranges). This means that granting Non-secure access to the <code>PADS</code> registers in the QFN-60 package does not allow Non-secure software to control the correct pads. It may also allow Non-secure control of pads that are not granted in <code>GPIO_NS_MASK0</code>. Note that the QSPI <code>PADS</code> registers (Bank 1) are not affected, as these are not remapped for different packages.
Workaround	Disable Non-secure access to the <code>PADS</code> registers by clearing <code>PADS_BANK0.NSP</code>, <code>NSU</code>. Implement a Secure Gateway (Arm) or <code>ecall</code> handler (RISC-V) to permit Non-Secure/U-mode code to read/write its assigned <code>PADS_BANK0</code> registers.
Fixed by	Documentation, software

Bootrom

RP2350-E10

Reference	RP2350-E10
Summary	UF2 drag & drop doesn't work with partition tables
Affects	RP2350 A2
Description	When dragging & dropping a UF2 onto the USB Mass Storage Device, the bootrom on chip revision A2 does not set up the flash before checking the partition table. This causes the UF2 download to fail if there is a partition table present.

Workaround	Add a single block at the start of the UF2 with an Absolute family ID, targeting the end of Flash, with block number set to 0 and number of blocks set to 2. This block will be written to flash first but doesn't reboot the device, and sets up the flash for the rest of the UF2 to be downloaded correctly. This is handled for you automatically by <code>picotool</code> in the SDK, which adds this block when generating UF2s if the <code>--abs-block</code> flag is specified.
Fixed by	Documentation, software

RP2350-E13

Reference	RP2350-E13
Summary	A binary containing both RP2040 and RP2350 <code>IMAGE_DEFS</code> (in that order) fails to boot
Affects	RP2350 A2
Description	When the block loop of a binary contains an <code>IMAGE_DEF</code> for RP2040 followed by an <code>IMAGE_DEF</code> for RP2350 (such as a universal binary), booting from that binary fails.
Workaround	Instead of an RP2040 <code>IMAGE_DEF</code>, use an <code>IGNORED</code> item. RP2040 does not require an <code>IMAGE_DEF</code> to boot a binary. SDK uses this workaround. When you set <code>PICO_CRT0_INCLUDE_PICOBIN_BLOCK</code>, the SDK uses an <code>IGNORED</code> item instead of an <code>IMAGE_DEF</code> for RP2040 binaries. You can override this behaviour and use an <code>IMAGE_DEF</code> by setting <code>PICO_CRT0_INCLUDE_PICOBIN_IMAGE_TYPE_ITEM</code>. For an additional example, see the universal binaries in <code>pico-examples</code>.
Fixed by	Documentation, software

RP2350-E14

Reference	RP2350-E14
Summary	The bootrom <code>connect_internal_flash()</code> function always uses pin 0, ignoring any configured <code>FLASH_DEVINFO</code> CS1 chip select pin
Affects	RP2350 A2
Description	When using the bootrom function <code>connect_internal_flash()</code> to configure CS1 (for instance, during a flash boot), the bootrom always configures the pad registers for pin 0, ignoring any CS1 pin specified in <code>FLASH_DEVINFO</code>. As a result, the specified CS1 pin remains isolated (see Section 9.7). Accesses to the QSPI device connected to the second chip select fails unless CS1 is connected to pin 0. <code>FLASH_DEVINFO</code> can be configured in OTP or at runtime. For more information, see flash_devinfo16_ptr.
Workaround	Manually configure the CS1 pads registers to remove the isolation after using the bootrom <code>connect_internal_flash()</code> function. Alternatively, connect CS1 to pin 0.
Fixed by	Documentation, software

RP2350-E15

Reference	RP2350-E15
Summary	The bootrom <code>otp_access()</code> function applies incorrect access permission to pages 62 & 63

Affects	RP2350 A2
Description	The bootrom <code>otp_access()</code> function incorrectly applies the access permissions specified in OTP rows <code>PAGE62_LOCK1</code> and <code>PAGE63_LOCK1</code> to the entirety of their respective OTP pages (62 and 63). This is incorrect, as pages 62 and 63 contain lock words for other pages: each lock word is instead protected by the permissions of the corresponding page. The ATE programming then locks down write access from non-Secure software and the bootloader to the page 63 lock word (to prevent non-Secure setting of the RMA flag), and write access from non-Secure software to the page 62 lock word. This prevents non-Secure software from modifying any of the OTP page locks, and the bootloader from modifying the locks for pages 32-63.
Workaround	When running code on the device, don't use the non-Secure <code>otp_access()</code> function to set locks for OTP pages. To set OTP page locks from non-Secure code, implement your own Secure API to do this that can be called from non-Secure code. Page locks for OTP pages 32-63 can be set by <code>picotool</code> using the <code>picotool otp permissions</code> command. This command loads a Secure binary into XIP SRAM on the device to change the permissions before rebooting back into the USB bootloader.
Fixed by	Documentation, software

RP2350-E18

Reference	RP2350-E18
Summary	The RP2350 will forever fail to boot if <code>FLASH_PARTITION_SLOT_SIZE</code> contains an invalid ECC bit pattern
Affects	RP2350 A2
Description	If ECC row programming is interrupted, an ECC row may contain a value which fails ECC validation. Because any ECC could potentially contain an invalid, partially written value, the bootrom uses a separate "enable" flag in OTP to indicate whether a particular ECC row is expected to contain a valid value. The user is expected to only set this flag <i>after</i> a particular ECC row is known to have been written correctly. For <code>FLASH_PARTITION_SLOT_SIZE</code>, the "enable" flag is <code>BOOT_FLAGS0.OVERRIDE_FLASH_PARTITION_SLOT_SIZE</code>. In the case of <code>FLASH_PARTITION_SLOT_SIZE</code>, the bootrom reads the row value and asserts the value is valid before checking the enable flag, and thus the boot process will hang if the <code>BOOT_FLAGS0.OVERRIDE_FLASH_PARTITION_SLOT_SIZE</code> row in OTP contains an invalid ECC value.
Workaround	Don't program <code>FLASH_PARTITION_SLOT_SIZE</code> or at least be aware that doing so may brick your device if the programming operation is interrupted or fails.
Fixed by	Documentation

RP2350-E19

Reference	RP2350-E19
Summary	RP2350 reboot hangs if certain bits are set in <code>FRCE_OFF</code> when rebooting.
Affects	RP2350 A2
Description	An incorrect assertion in the boot path, assumes the all bits (other than <code>FRCE_OFF.PROC1</code>) are clear. These bits can only be set during boot if the user had set them and then re-entered the boot path.
Workaround	Do not perform a <code>WATCHDOG</code> or <code>POWMAN</code> boot, or a core0 reset with bits other than <code>FRCE_OFF.PROC1</code> set.

Fixed by	Documentation
----------	---------------

RP2350-E20

Reference	RP2350-E20
Summary	An attacker with physical access to the chip and the ability to physically "glitch" the CPU at precise times, could cause unsigned code execution on a secured RP2350 by targeting legitimate non-Secure calls to the bootrom <code>reboot()</code> function
Affects	RP2350 A2
Description	The RP2350 bootrom provides a <code>reboot()</code> function to reboot the RP2350. This method is potentially accessible to non-Secure callers via the <code>PICOB00T</code> interface (e.g. <code>picotool</code>) or, if the corresponding permission is set, to non-Secure code running on the device. A particular reboot type (<code>REBOOT_TYPE_PC_SP</code>) is not allowed in the bootrom <code>reboot()</code> function when called from non-Secure code as it launches user-provided code in a Secure state post reboot. The <code>reboot()</code> function correctly disallows this reboot type when called from a non-Secure context, however if instead a valid reboot type (e.g. <code>REBOOT_TYPE_NORMAL</code>) is passed to the function, a later precisely-timed processor glitch can cause an incorrect code path to be taken which ends up configuring the WATCHDOG scratch registers in such a way that allows secure execution of user provided code post reboot.
Workaround	If any <code>WATCHDOG</code> based reboot types other than into the regular boot path are not required (note this includes programmatic reboots into <code>BOOTSEL</code> mode and <code>FLASH_UPDATE</code> boots which are important when using A/B partitions), the OTP flag <code>BOOT_FLAGS0.DISABLE_WATCHDOG_SCRATCH</code> can be set, which causes the WATCHDOG scratch registers to be completely ignored during boot, meaning that the only type of boot available via WATCHDOG reset is regular boot. A more refined approach would be to disable use of the <code>reboot()</code> function from non-Secure code. This is the default case anyway for non-Secure code started by a secure application (see Section 5.4.2), however the <code>BOOTSEL</code> mode bootloader is itself non-Secure application which does have access to the function. <code>BOOTSEL</code> mode however can be disabled if not needed via <code>BOOT_FLAGS0.DISABLE_BOOTSEL_UART_BOOT</code>, <code>BOOT_FLAGS0.DISABLE_BOOTSEL_USB_PICOB00T_IFC</code>, and <code>BOOT_FLAGS0.DISABLE_BOOTSEL_USB_MSD_IFC</code>. Note that <code>BOOT_FLAGS0.DISABLE_BOOTSEL_USB_PICOB00T_IFC</code> is the most important, as <code>PICOB00T</code> provides a conduit for a user to pass specific parameters to the bootrom <code>reboot()</code> function, however any other use of <code>BOOTSEL</code> mode could be vulnerable in conjunction with some other future attack on the non-Secure code.
Credit	Marius Muench
Fixed by	Documentation

RP2350-E21

Reference	RP2350-E21
Summary	An attacker with physical access to the chip, and the ability to physically "glitch" the CPU at precise times, could potentially extract sensitive data from OTP on a RP2350 in <code>BOOTSEL</code> mode.
Affects	RP2350 A2

Description	An attacker with physical access to the chip, could precisely time physical glitch attacks during entry to BOOTSEL mode and cause some OTP page access permissions for BOOTSEL mode not to be applied, leading to possible exposure of sensitive data. The RP2350 BOOTSEL mode exposes an API over PICOBOOT such that a user can read/write OTP rows via <code>picotool</code>. Of course, certain OTP rows (e.g. encryption keys or other secrets) should not be readable via this method. Equally, certain rows might be protected against writes. This is handled at the page (64 row) level by page locks stored in OTP. On entry to BOOTSEL mode, the OTP should be locked down such that no software (including Secure software) can access anything not marked as accessible to BOOTSEL mode. However, with two precisely timed processor glitches, it is possible to prevent a page being correctly locked.
Workaround	1. Disable the BOOTSEL mode bootloader altogether via <code>BOOT_FLAGS0.DISABLE_BOOTSEL_UART_BOOT</code>, <code>BOOT_FLAGS0.DISABLE_BOOTSEL_USB_PICOBOOT_IFC</code>, and <code>BOOT_FLAGS0.DISABLE_BOOTSEL_USB_MSD_IFC</code>. Note that <code>BOOT_FLAGS0.DISABLE_BOOTSEL_USB_PICOBOOT_IFC</code> is the most important, as PICOBOOT provides the conduit for a user to access the OTP, however any other use of BOOTSEL mode could be vulnerable in conjunction with some other future attack on the non-Secure code. 1. Use an OTP access key (Section 13.5.2) to protect OTP data you do not want accessed from BOOTSEL mode, although this only helps if the data is not needed until your application can provide the key.
Credit	Thomas Roth
Fixed by	Documentation

RP2350-E22

Reference	RP2350-E22
Summary	Parsing a malformed "lollipop" block loop will cause a hang rather than a failure
Affects	RP2350 A2
Description	<code>PARTITION_TABLES</code> and <code>IMAGE_DEFS</code> metadata are stored as part of a block loop. These block loops are parsed during boot and at other times. A "lollipop" block loop is an invalid block loop, which loops back from the last block to a block which is not the first. Such an invalid block loop is never generated by the SDK or by <code>picotool</code>, however could potentially be generated by other tooling.
Workaround	Don't use "lollipop" block loops. If you program a "lollipop" block loop into flash such that it is read during the boot process, it will cause a boot hang and also a hang on entry into BOOTSEL mode. Therefore, to re-enable booting, you will have to clear the flash in some other way, for example from the debugger over SWD.
Fixed by	Documentation

RP2350-E23

Reference	RP2350-E23
Summary	PICOBOOT <code>GET_INFO</code> command always returns zero for <code>PACKAGE_SEL</code>
Affects	RP2350 A2

Description	The PICOB00T GET_INFO command can be used to get system information in a similar way to the bootrom get_sys_info() function. This information can include the value read from PACKAGE_SEL which indicates whether the RP2350 package is QFN60 or QFN80. The SYSINFO block is erroneously left in reset when entering BOOTSEL mode, and thus this register is read as zero when accessed via PICOB00T. Note this problem does not affect use of the bootrom get_sys_info() function itself from user code.
Workaround	Determine the package size by reading register_link_macro:[register=OTP_DATA_NUM_GPIOS_ROW] via the PICOB00T OTP_READ command instead. This workaround is used by picotool .
Fixed by	Documentation, Software

RP2350-E24

Reference	RP2350-E24
Summary	An attacker with physical access to the chip, moderate hardware, and the ability to physically "glitch" the CPU at precise times, could cause unsigned code execution on a secured RP2350.
Affects	RP2350 A2
Description	An attacker with physical access to the chip, and the ability to switch the contents of "flash" as read by the RP2350 over QSPI during boot at precise times, could, combined with a precisely-timed physical "glitch" attack of the CPU, trick the bootrom into checking the signature of data other than the program binary as loaded into memory during secure boot. If this "other" data passes the signature check, then the attacker's binary is executed without itself having passed a signature check, which allows the user to run arbitrary unsigned code on the RP2350.
Credit	Kévin Courdresses (see https://courk.cc/rp2350-challenge-laser#flash-memory-organization)
Workaround	none
Fixed by	not fixed

RP2350-E25

Reference	RP2350-E25
Summary	A LOAD_MAP which uses non-word sizes does not cause an error.
Affects	RP2350 A2
Description	Non-word sizes in a LOAD_MAP are not supported and were documented as such, however they do not currently cause an error. Whilst non-word sizes may currently work in some certain cases, you should never use them, as they may not work as you expect in all cases and may be properly treated as an error in the future. Note that the SDK and picotool do not generate such LOAD_MAPs with non-word sized entries.
Workaround	Do not use non-word (not multiple of 4) sizes in a LOAD_MAP . A best practice is to make sure that linker memory segments are both word-sized and word-aligned.
Fixed by	Documentation

DMA

RP2350-E5

Reference	RP2350-E5
Summary	Interactions between CHAIN_TO and ABORT of active channels
Affects	RP2350 A2
Description	The CHAN_ABORT register commands a DMA channel to stop issuing transfers, and to clear its BUSY flag once in-flight transfers have completed. This was originally intended for recovering channels which are stuck with their DREQ low. An ABORT is initiated by writing a bitmap of aborted channels to CHAN_ABORT. Bits remain set until each channel comes to rest. This erratum is a compound of two behaviours: first, aborting a channel will cause its CHAIN_TO to fire, if and only if the aborted channel is the last channel to have completed a write transfer. Second, a channel undergoing an ABORT is susceptible to be re-triggered on the last cycle before the ABORT register clears, because the channel is both inactive and enabled on this cycle, and the ABORT itself does not inhibit triggering. However, since the ABORT is still in effect, the transfer count is held at zero. On the cycle after the ABORT finishes, the channel completes because its transfer counter is zero. This causes the channel's IRQ and CHAIN_TO to fire on the cycle after the ABORT completes. These two behaviours are problematic when aborting multiple channels that chain to one another, since they may cause the channels to immediately restart post-abort.
Workaround	Before aborting an active channel, clear the EN bit (CH0_CTRL_TRIG.EN) of both the aborted channel and any channel it chains to. This ensures the channel is not susceptible to re-triggering.
Fixed by	Documentation

RP2350-E8

Reference	RP2350-E8
Summary	CHAIN_TO may not fire for zero-length transfers
Affects	RP2350 A2
Description	The CTRL.CHAIN_TO field configures a channel to start another channel once it completes its programmed sequence of transfers. The CHAIN_TO takes place on the cycle where the channel's last write completes, and the chaine becomes active on the next cycle. The hardware implementation assumes that CHAIN_TO always happens as a result of a write completion. This is not the case when a channel is triggered with a transfer count of zero: in this case the channel completes on the cycle immediately after the trigger, without performing any bus accesses. A CHAIN_TO from a channel started with a transfer count of zero will fire if and only if that channel is the last channel to have completed a write transfer. This is true only when the channel in question has previously performed a non-zero-length sequence of transfers, and no other channel has completed a write since.
Workaround	Do not use CHAIN_TO in conjunction with zero-length transfers. Avoid zero-length transfers in the middle of control block lists, and replace them with dummy transfers if possible.
Fixed by	Documentation

GPIO

RP2350-E9

Reference	RP2350-E9
Summary	Increased leakage current on Bank 0 GPIO when pad input is enabled
Affects	RP2350 A2

Description	For GPIO pads 0 through 47: Increased leakage current when Bank 0 GPIO pads are configured as inputs and the pad is somewhere between V_{IL} and V_{IH} (the undefined logic region). When the pad is set as an input (input enable is enabled and output enable is disabled) and the voltage on the pad is within the undefined logic region, the leakage current exceeds the standard specified I_{IN} leakage level. During this condition the pad can source current (the exact amount is dependent on the chip itself and the exact pad voltage, but typically around 120µA). This leakage will hold the pad at around 2.2 V as that is the effective source voltage of the leakage, and can only be overcome with a suitably low impedance driver / pull. Note that the pad pull-down (if enabled) is significantly weaker than the leakage current in this state and therefore is not strong enough to pull the pad voltage low. Driving / pulling the pad input low with a low impedance source of 8.2 kΩ or less will overcome the erroneous leakage and drive the voltage below the level where the leakage current occurs, so in this case if the pad is driven / pulled low it will stay low. The erroneous leakage only occurs (and continues to occur) when the pad input enable is enabled; disabling the input enable will reset (remove) the leakage. The pad pull-up still works. If enabled it will pull the pad to IOVDD as it will pull the input voltage out of the problematic range. The voltages and currents above are based on IOVDD at 3.3 V. For IOVDD at 1.8 V the effective source voltage of the leakage becomes 1.8 V and the peak current is around 30µA. This is effectively a pull-up (separate to the standard pad pull-up) when the pad voltage is between 0.6 V and 1.8 V. These graphs show the leakage current versus pad input voltage for a typical chip for IOVDD at 3.3 V Figure 152 and 1.8 V Figure 153. In detail, this issue presents under the following conditions, for any GPIO 0 through 47: 1. The voltage on the pad is in the undefined logic region. 2. Input buffer is enabled in GPIO0.IE 3. Output buffer is disabled (e.g. selecting the NULL GPIO function) 4. Isolation is clear in GPIO0.ISO, or the previous were true at the point isolation was set When all of the above conditions are met, the input leakage of the pad may exceed the specification. This issue may affect a number of common circuits: • Relying on floating pins to have a low leakage current • Relying on the internal pull-down resistor If the internal pull-up is enabled then any floating signal will be pulled high thus removing increased leakage condition as the excess leakage is only sourcing current. This of course can't prevent the increased leakage if the pad is fed via a strong source e.g. strong potential divider. Note that this does not affect the pull-down behaviour of the pads immediately following a PoR or RUN reset, because the input enable field is initially clear. The pull-down resistor functions normally in this state. This issue does not affect the QSPI pads, which use a different pad macro without the faulty circuitry. The USB PHY's pins are also unaffected. This issue does also affect the SWD pads, which use the same fault-tolerant pad macro as the Bank 0 GPIOs. However, both SWD pads are pull-up by default, so there is no ill effect.
-------------	--

Workaround	If pad pull-down behaviour is required, clear the pad input enable in GPIO0.IE (for GPIOs 0 through 47) to ensure that the pad pull-down resistor pulls the pad signal low. To read the state of a pad pulled-down GPIO from software, enable the input buffer by setting GPIO0.IE immediately before reading, and then re-disable immediately afterwards. Note that if the pad is already a logic-0, re-enabling the input does not disturb the pull-down state. Alternatively an external pull-down of 8.2 kΩ or less can be used. Note that PIO programs can't toggle pad controls and therefore external pulls may be required, depending on your application. As normal, if ADC channels are being used on a pin, clear the relevant GPIO input enable as stated in Section 12.4.3.
Fixed by	Documentation

Figure 152. GPIO Pad leakage for IOVDD=3.3 V

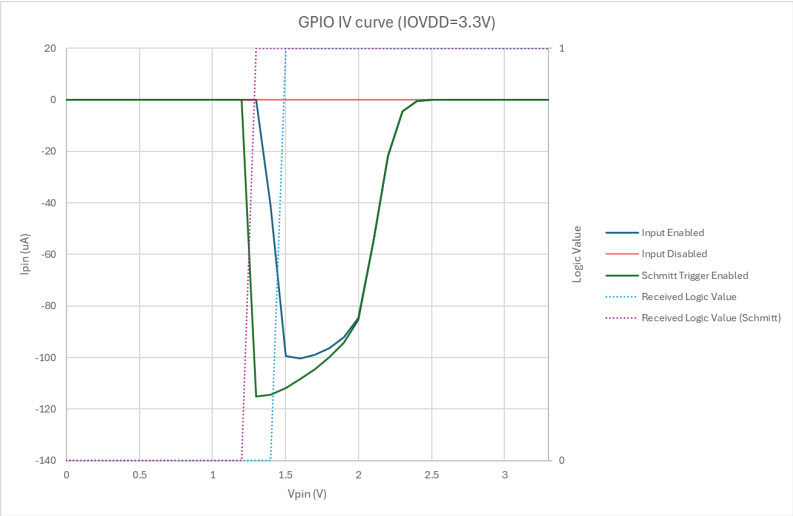
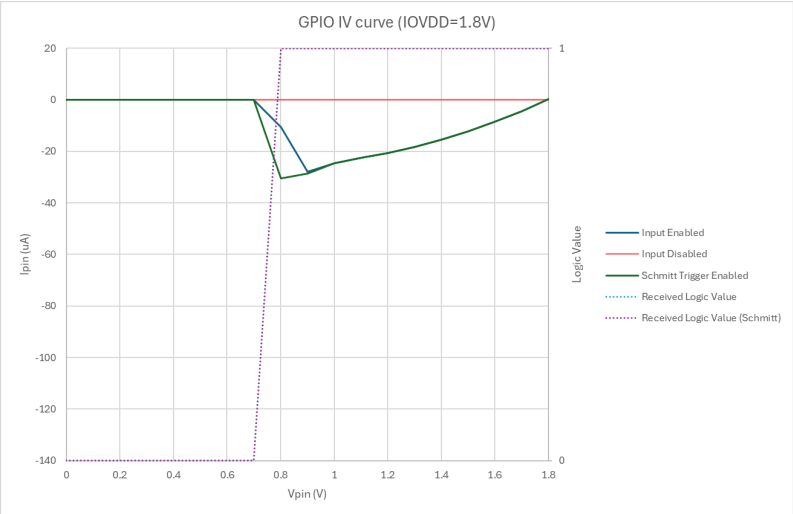


Figure 153. GPIO Pad leakage for IOVDD=1.8 V



Hazard3

RP2350-E4

Reference	RP2350-E4
Summary	System Bus Access stalls indefinitely when core 1 is in clock-gated sleep
Affects	RP2350 A2
Description	System Bus Access (SBA) is a RISC-V debug feature that allows the Debug Module direct access to the system bus, independent of the state of harts in the system. RP2350 implements SBA by arbitrating Debug Module bus accesses with the core 1 load/store port. Hazard3 implements custom low-power states controlled by the MSLEEP CSR. When MSLEEP.DEEPSLEEP is set, Hazard3 completely gates its clock, with the exception of the minimal logic required to wake again. Due to a design oversight, this also clock-gates the arbiter between SBA and load/store bus access. (This is addressed in upstream commit c11581e.) Consequently, if you initiate an SBA transfer whilst MSLEEP.DEEPSLEEP is set on core 1, and core 1 is in a WFI-equivalent sleep state, the SBA transfer will make no progress until core 1 wakes from the WFI state. The processor wakes upon an enabled interrupt being asserted, or a debug halt request.
Workaround	Either configure your debug translator to not use SBA, or do not enter clock-gated sleep on core 1. The A2 bootrom mitigates this issue by not setting DEEPSLEEP in the initial core 1 wait-for-launch code. Note that the processors are synthesised with hierarchical clock gating, so the top-level clock gate controlled by the DEEPSLEEP flag brings minimal power savings over a default WFI sleep state.
Fixed by	Documentation

RP2350-E6

Reference	RP2350-E6
Summary	PMPCFGx RWX fields are transposed
Affects	RP2350 A2
Description	The Physical Memory Protection unit (PMP) defines read, write and execute permissions (RWX) for configurable ranges of physical memory. The RWX permissions for four regions are packed into each 32-bit PMPCFG register, PMPCFG0 through PMPCFG3. Per the RISC-V privileged ISA specification, the permission fields are ordered X, W, R from MSB to LSB. Hazard3 implements them in the order R, W, X. This means software using the correct bit order will have its read permissions applied as execute, and vice versa. (See upstream commit 7d37029.)
Workaround	When configuring PMP with $X \neq R$, use the bit order implemented by this version of Hazard3. In the SDK, the hardware/regs/rvcsr.h register header provides bitfield definitions for the as-implemented order when building for RP2350.
Fixed by	Documentation

RP2350-E7

Reference	RP2350-E7
Summary	U-mode does not ignore mstatus.mie
Affects	RP2350 A2

Description	The MSTATUS.MIE bit is a global enable for interrupts which target M-mode. Software generally clears this momentarily to ensure short critical sections are atomic with respect to interrupt handlers. The RISC-V privileged ISA specification requires that the interrupt enable flag for a given privilege mode is treated as 1 when the hart is in a lower privilege mode. In this case, <code>mstatus.mie</code> should be treated as 1 when the core is in U-mode. Hazard3 does not implement this rule, so entering U-mode with M-mode interrupts disabled results in no M-mode interrupts being taken. (See upstream commit a84742a.)
Workaround	When returning to U-mode from M-mode via an <code>mret</code> with <code>mstatus.mpp == 0</code> , ensure <code>mstatus.mpie</code> is set, so that IRQs will be enabled by the return.
Fixed by	Documentation

OTP

RP2350-E16

Reference	RP2350-E16
Summary	<code>USB_OTP_VDD</code> disruption can result in corrupt OTP row read data
Affects	RP2350 A2
Description	The OTP array has a read voltage generated from <code>USB_OTP_VDD</code> using an internal linear regulator. While the regulator has a "power good" signal, it is not sampled outside of the initial power-on reset startup sequence. External manipulation of <code>USB_OTP_VDD</code> can result in incorrect data being latched during the array read phase. The erroneous behaviour includes, but is not limited to: • Latching the previous read cycle data from the array • One or many bitlines returning zeroes for programmed bits • Byte-shifted read data In the case of guarded reads, the first failure mode can result in the guard read check passing and the guard word also ending up as the read data. If the critical data are the <code>CRIT0/CRIT1</code> flags, sampled by the OTP PSM during boot, this can enable Hazard3 debug and disable the Arm cores which results in a reversion of the effects of the <code>CRIT1.SECURE_BOOT_ENABLE</code> and <code>CRIT1.DEBUG_DISABLE</code> flags. Guarded ECC reads are not typically vulnerable to corruption of this nature as the guard word is an invalid ECC word, and bit deletion or byte shifting reliably invalidates the ECC check.
Credit	Aedan Cullen (see https://github.com/aedancullen/hacking-the-rp2350)
Workaround	None
Fixed by	Not fixed

RP2350-E17

Reference	RP2350-E17
------------------	------------

Summary	Performing a guarded read on a single ECC OTP row causes a fault if the data in the adjacent row is not also valid ECC data.
Affects	RP2350 A2
Description	Each "ECC row" in OTP stores 16 bits of user data along with error correction information used to correct and/or detect bit errors. ECC rows are used to store data values which are written a full 16 bits at a time into OTP. A "guarded" ECC row read is intended to be used by the RP2350 boot path, or other Secure software when it expects to read an ECC row and cannot proceed if the row value is invalid. Reading such an invalid ECC value via a "guarded" read will halt the chip until it is rebooted. If ECC row programming is interrupted, an ECC row may contain a value which fails ECC validation. Because any ECC could potentially contain an invalid, partially written value, the bootrom uses a separate "enable" flag in OTP to indicate whether a particular ECC row is expected to contain a valid value. The user is expected to only set this flag <i>after</i> a particular ECC row is known to have been written correctly. The RP2350 OTP hardware actually reads a pair of rows (starting on the even row) whenever an ECC read is performed but only returns one row value. When performing a guarded ECC read, it actually checks both rows validity, so the guarded read can cause a halt if either row in the pair is not a valid ECC value.
Workaround	• Never store ECC rows and RAW rows in the same pair of rows (a pair of rows starting on an even row number), since the RAW row is unlikely to always contain a valid ECC value. Note however that zero in a RAW row is a valid ECC value. • Never store two ECC rows in the same pair of rows if they are protected by different "enable" flags. This workaround is fine for user use of OTP, however certain pre-existing ECC row pairs used by the bootrom violate workaround 2: • FLASH_DEVINFO and FLASH_PARTITION_SLOT_SIZE • BOOTSEL_LED_CFG and BOOTSEL_PLL_CFG To be absolutely safe, you should not update and set the "enable" flag for one half of the pair after you have set the "enable" flag for the other half. If you want to set both ECC values safely, set them both, then set both "enable" flags.
Fixed by	Documentation

RCP

RP2350-E26

Reference	RP2350-E26
Summary	RCP random delays can create a side-channel
Affects	RP2350 A2

Description	The RCP delay is implemented as a coprocessor stall; this has the effect of completely pausing the associated core. As the core is effectively halted for the duration of the delay, this represents a significant reduction in gate toggle activity across the chip if there are no other bus managers active (e.g. other CPU or DMA). The reduction in toggle activity causes a reduction in DVDD current, and the typical length of the delay means that the reduction is measureable outside of the chip. The reduction in current and subsequent increase may create a fault injection trigger point. Instructions immediately after an RCP delay operation can be more reliably targeted, undoing the cumulative effect of clock randomisation. A second-order effect of the RCP delay probability distribution is that after N RCP instructions for large N, the added latency converges to a normal distribution centred on $N * 63$ cycles. Therefore instructions after a known number of RCP delays are statistically easier to target. With these two factors in mind, programmers should use RCP delays in Secure code with great care. In particular, avoid using RCP delays: • Inside inner loops that may be executed many times • As part of boilerplate assembly in function prologues/epilogues • Immediately prior to particularly critical actions, such as modifying <code>ACCESSCTRL</code>.
Workaround	Use of the non-delay RCP instruction variant is recommended.
Fixed by	Documentation, Software

SIO

RP2350-E1

Reference	RP2350-E1
Summary	Interpolator <code>OVERF</code> bits are broken by new right-rotate behaviour
Affects	RP2350 A2
Description	RP2350 replaces the interpolator right-shift with a right-rotate, so that left shifts can be synthesised. This is useful for scaled indexed addressing in tight address-generating loops. The <code>OVERF</code> flag functions by checking for nonzero bits in the post-shift value that have been masked out by the <code>MSB</code> mask configured by the <code>CTRL_LANE0_MASK_MSB</code> and <code>CTRL_LANE1_MASK_MSB</code> register fields. This is used to discard samples outside of the $[0, 1)$ wrapping domain of UV coordinates represented by <code>ACCUM0</code> and <code>ACCUM1</code>, for example in affine-transformed sprite sampling. The issue occurs because the right-rotate causes nonzero LSBs to be rotated up to the MSBs. These nonzero bits spuriously set the <code>OVERF</code> flag.
Workaround	Either compute <code>OVERF</code> manually by checking the <code>ACCUM0/ACCUM1</code> MSBs, or precompute the bounds in advance to avoid per-sample checks.
Fixed by	Documentation

RP2350-E2

Reference	RP2350-E2
Summary	SIO SPINLOCK writes are mirrored at +0x80 offset

Affects	RP2350 A2
Description	The SIO contains spinlock registers, SPINLOCK0 through SPINLOCK31. Reading a spinlock register attempts to claim it, returning nonzero if the claim was successful and 0 if unsuccessful. Writing to a spinlock register releases it, so the next claim will be successful. SIO spinlock registers are at register offsets 0x100 through 0x17c within SIO. RP2350 adds new SIO registers at register offsets 0x180 and above: Doorbells, the PERI_NONSEC register, the RISC-V soft IRQ register, the RISC-V MTIME registers, and the TMDS encoder. The SIO address decoder detects writes to spinlocks by decoding on bit 8 of the address. This means writes in the range 0x180 through 0x1fc are spuriously detected as writes to the corresponding spinlock address 128 bytes below, in the range 0x100 through 0x17c. Writing to any of these high registers will set the corresponding lock to the unclaimed state. This erratum only affects writes to the spinlock registers. Reads are correctly decoded, so are not affected by accesses above 0x17c.
Workaround	Use processor atomic instructions instead of the SIO spinlocks. The SDK hardware_sync_spin_lock library uses software lock variables by default when building for RP2350, instead of hardware spinlocks. The following SIO spinlocks can be used normally as they do not alias with writable registers: 5, 6, 7, 10, 11, and 18 through 31. Some of the other lock addresses may be used safely depending on which of the high-addressed SIO registers are in use. Note that locks 18 through 24 alias with some read-only TMDS encoder registers, which is safe as only writes are misdecoded.
Fixed by	Documentation, software

XIP

RP2350-E11

Reference	RP2350-E11
Summary	XIP cache clean by set/way operation modifies the tag of dirty lines
Affects	RP2350 A2

Description	The 0x1 clean by set/way cache maintenance operation performs the following steps: 1. Selects a cache line: address bits 12:3 index the cache sets, and bit 13 selects from the two 8-byte cache lines which make up the ways of each set 2. Checks if the line contains uncommitted write data (a dirty line) 3. If the line is dirty, writes the data downstream and marks the line as clean In the third step, in addition to marking the line as clean, the cache controller erroneously sets the cache line's tag to address bits 25:13 of the maintenance write which initiated the clean operation. The cache uses the tag to recall which of the many possible downstream addresses currently resides in each cache line. Therefore reading the newly tagged address returns cached data from the original address, breaking the memory contract. Consider the following example scenario: • QMI window 0 (starting at 0x10000000) has a flash device attached • QMI window 1 (starting at 0x11000000) has a PSRAM device attached • The cache possesses address 0x11000000 in the dirty state, and it is allocated in way 0 of the cache The programmer cleans the cache, starting by writing to address 0x18000001 to clean set 0, way 0. This cleans the dirty line containing address 0x11000000. After cleaning, the cache updates this line's tag to all-zeroes (the offset of the maintenance write). A subsequent read from 0x10000000 results in a spurious cache hit, returning PSRAM data in place of flash data. See Section 4.4.1.1 for more information about cache maintenance operations. See Section 4.4.1.2 for more information about cache line states and state transitions. The tag update <i>only</i> affects 0x1 clean by set/way; is either correct or harmless for the other four cache maintenance operations.
Workaround	To avoid spurious cache hits, choose an address that cannot alias with cached data from the QMI. This remaps dirty lines outside of the QMI address space after cleaning them, which has the side effect of causing a cache miss on the next access to the dirty address. The SDK <code>xip_cache_clean_all()</code> function implements this workaround. The updated tag is predictable: it is always the address of the maintenance write. For example, use the upper 16 kB of the maintenance space to clean all cache lines: <pre>1 volatile uint8_t *maintenance_ptr = (volatile uint8_t*)0x1bffc001u; 2 for (int i = 0; i < 0x4000; i += 8) { 3 maintenance_ptr[i] = 0; 4 }</pre> Because the clean operation is a no-op for invalid, clean or pinned lines, this workaround does not interfere with lines pinned for cache-as-SRAM use.
Fixed by	Documentation, software

USB

RP2350-E12

Reference	RP2350-E12
-----------	------------

Summary	Inadequate synchronisation of USB status signals
Affects	RP2350 A2
Description	Within the USB peripheral, certain Host and Device controller events cross from <code>clk_usb</code> to <code>clk_sys</code>. Many of these signals do not have appropriate synchronisation methods to ensure that they are correctly registered when <code>clk_sys</code> is equal to or slower than <code>clk_usb</code>. The following signals lack appropriate synchronisation methods: SIE_STATUS: • <code>TRANS_COMPLETE</code> • <code>SETUP_REC</code> • <code>STALL_REC</code> • <code>NAK_REC</code> • <code>RX_SHORT_PACKET</code> • <code>ACK_REQ</code> • <code>DATA_SEQ_ERROR</code> • <code>RX_OVERFLOW</code> INTR: • <code>HOST_SOF</code> • <code>ERROR_CRC</code> • <code>ERROR_BIT_STUFF</code> • <code>ERROR_RX_OVERFLOW</code> • <code>ERROR_RX_TIMEOUT</code> • <code>ERROR_DATA_SEQ</code> The bootrom's USB bootloader chains <code>clk_sys</code> from <code>clk_usb</code>, therefore the two clock frequencies are identical and have a fixed phase relationship. In this condition and at extremes of PVT, lab testing has observed that these events may be lost, which results in unreliable USB bootloader behaviour.
Workaround	Run <code>clk_sys</code> faster than <code>clk_usb</code> by at least 10% when the peripheral is in use. Signalling of quasi-static bus states such as reset, suspend, and resume are not affected by this erratum, so <code>clk_sys</code> can be lower in these cases.
Fixed by	Documentation, Software